Predicting Gata4 HT-SELEX affinities from MD-derived DNA shape: preprocessing and model comparison with new tetramer/hexamer shape tables

Pieter Zanders

22 October 2026

Abstract

We characterise newly generated tetramer and hexamer DNA-shape tables (average helical parameters and stiffness matrices) and use them to predict Gata4 HT-SELEX 10-mer affinities. All labels come from the SELEX affinity table (`4.txt`), in which affinity is exactly $0.172\,(\text{obs}/\text{exp})^{1/4}$, only 12.1% of 10-mers are present and 52.5% of entries have a single count. We ran a preprocessing ablation under a leakage-free validation scheme (5-fold cross-validation grouping each k-mer with its reverse complement, plus a fixed 10% held-out test set). Four choices worked best: log-affinity labels, a count threshold of $\geq$10, per-strand labels (no averaging of the two orientations) and tetramer shape features plus per-window one-hot encoding with standardisation. Per-strand labels gained 0.15–0.17 in CV R^2 over merged labels. Eight regressors (Linear, Ridge, SVR, random forest, DFN, CNN1D, CNNAttention, Transformer) were then benchmarked on four feature sets: tetramer, hexamer or tetramer+hexamer shape (each with one-hot), and one-hot only. With tetramer shape + one-hot, CNN1D was the best model (held-out test $R^2 = 0.736 \pm 0.012$, Pearson 0.860, Spearman 0.844; 0.788 for the averaged prediction of its 15 runs), followed by CNNAttention (0.725), the Transformer (0.685), DFN (0.644), SVR (0.598), random forest (0.572) and linear/ridge regression (0.47). Without any shape features the convolutional models were equally good (CNN1D 0.739, CNNAttention 0.729), whereas shape features improved the models without a positional sequence bias: Linear +0.09, RF +0.036, SVR +0.017, DFN +0.009. Hexamer tables never beat tetramer tables for non-linear models. The supplied `SELEX_test.txt` is not consistent with `4.txt` and was not used for evaluation.

1 Introduction

Transcription factors (TFs) recognise their genomic targets through direct base readout and through the sequence-dependent three-dimensional shape of DNA ("shape readout") [2]. High-throughput SELEX (HT-SELEX) enriches DNA ligands bound by a TF over successive selection rounds and sequences them, giving large-scale relative affinities for k-mers [1]. Physics-based descriptors from atomistic molecular dynamics (MD) summarise the local structure and flexibility of DNA. Examples are average base-step helical parameters (shift, slide, rise, tilt, roll, twist) and stiffness (force-constant) matrices. Such descriptors can complement or replace the raw sequence as model input, and they give an interpretable view of affinity determinants. DNAffinity builds regressors on such features. Here we test new tetramer- and hexamer-level shape tables on Gata4 HT-SELEX data and ask (i) which preprocessing is optimal and (ii) how classical and deep models compare.

2 Methods

2.1 Affinity labels

Labels are taken from the SELEX package output for Gata4 (`data/selex/Gata4/4.txt`). It has 126,624 10-mers with observed and expected (Markov-model) counts, affinity and SE. We verified that Affinity $= 0.172\,(\text{obs}/\text{exp})^{0.25}$ to machine precision (sd of the ratio 2.5×10^{-16}) and that $\text{SE}\sqrt{n}/\text{Aff}$ has median $1.414 \approx \sqrt{2}$. The table is zero-truncated: only 12.1% of the 4^{10} possible 10-mers appear. 52.5% of entries have count 1, and 13,100 have count $\geq$10. Because the SE is about $\sqrt{2}\,\text{Aff}/\sqrt{n}$, labels with low counts are very noisy.

2.2 Shape features

The new tables `data/selex/tetramer/{avg,fce}_tetramer.dat` and `data/selex/hexamer/{avg,fce}_hexamer.dat` give, for every tetramer (256) and hexamer (4096), six average helical parameters (`avg`) and a flattened 6×6 stiffness matrix (`fce`). The matrices are exactly symmetric and positive definite (minimum eigenvalue $\approx$ 0.006), so we keep the 21 unique upper-triangle entries. A 10-mer is represented by 7 overlapping tetramer windows (and 5 hexamer windows). Hexamer j is aligned to its central tetramer window $j{+}1$ and zero-padded at the ends for windowed models. Optional one-hot input adds the 16 one-hot values of the bases in each tetramer window. Flat models use the concatenated windows; windowed models (CNN1D, CNNAttention, Transformer) use a $7 \times d$ tensor.

2.3 Data processing pipeline

All steps are implemented in `scripts/selex_new_shape/lib.py` (functions `load_labels`, `split`, `flat_feats`, `windowed`, `tf`) and `train_final.py`. Figure 1 summarises them.

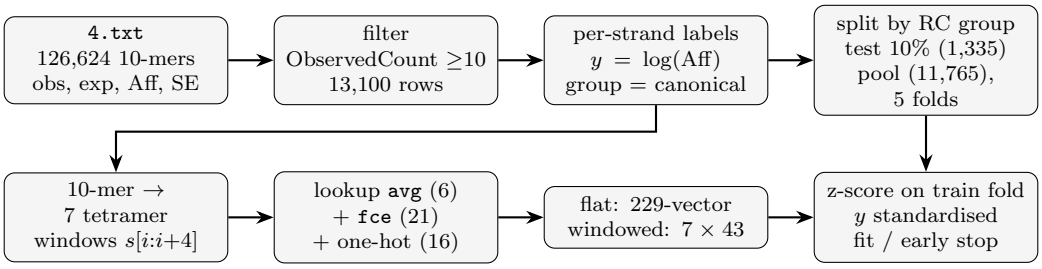


Figure 1: Processing pipeline for the chosen configuration. Row counts are those of the final runs.

2.3.1 Reading and checking the labels

`4.txt` (tab-separated, quoted header, R row names) is read with pandas. The columns used are `Kmer`, `ObservedCount` (n), `ExpectedCount` (e, from the Markov background model of round 0) and `Affinity`. We checked that

$$\text{Aff} = 0.172\,(n/e)^{1/4}, \qquad \text{SE} \approx \sqrt{2}\,\text{Aff}/\sqrt{n}, \tag{1}$$

so $\text{SE}/\text{Aff} = \sqrt{2/n}$: 141% at $n = 1$ and 45% at $n = 10$. The table is zero-truncated. Only k-mers seen at least once in the selected round are listed (12.1% of 4^{10}), and there are no entries for unobserved, low-affinity k-mers. The models therefore learn on the observed range only.

2.3.2 Count filter

Rows with $n \geq 10$ are kept (`load_labels(min_count=10)`), leaving 13,100 of 126,624 rows. This bounds the relative SE at about 45%. The ablation (Table 2) chose this threshold.

2.3.3 Per-strand vs. merged labels

Each row's group key is the canonical sequence $\min(s, \mathrm{rc}(s))$, with `canonical` from `dnaffinity.preprocess`.

- *Per-strand* mode (chosen) keeps each measured orientation as its own row with its own label. A reverse-complement row exists only if that orientation itself passed the filter. The 13,100 rows form 10,771 groups.
- *Merged* mode averages the affinities of the measured orientations per group and emits the canonical sequence and its reverse complement with the same label (reverse-complement augmentation).

2.3.4 Label transform

The target is $y = \log(\mathrm{Aff}) = \frac{1}{4}\log(n/e) + \log 0.172$, i.e. a scaled log-enrichment. Inside each training fold, y is also standardised for model fitting, $\tilde{y} = (y - \bar{y}_{\mathrm{tr}})/s_{\mathrm{tr}}$, and predictions are transformed back. All metrics are computed on y.

2.3.5 Windowing and shape lookup

A 10-mer s is cut into the 7 overlapping tetramers $s[i{:}i{+}4]$, $i = 0, \ldots, 6$ (each describes the central base step of its window). For each window we look up:

- the 6 averages of `avg_tetramer.dat` (shift, slide, rise, tilt, roll, twist);
- the 21 upper-triangle entries (including the diagonal) of the symmetric 6×6 stiffness matrix in `fce_tetramer.dat`. The 36 stored values contain 15 duplicates.

Hexamer tables, used only in the ablation, are read the same way. A flat vector concatenates the 5 hexamers $s[j{:}j{+}6]$. In windowed form, hexamer j is placed at tetramer position $j{+}1$, i.e. aligned on its central tetramer, and the two end positions are zero-padded.

2.3.6 One-hot encoding

Bases are encoded A,C,G,T $\rightarrow$ unit vectors of length 4. Flat models receive the one-hot of the full 10-mer ($10 \times 4 = 40$ values). Windowed models receive the one-hot of the 4 bases of each window (16 values per position).

2.3.7 Feature tensors per model

- **Flat** (Linear, Ridge, SVR, RF, DFN): $[\mathtt{avg}_{0..6}\,(42),\ \mathtt{fce}_{0..6}\,(147),\ \text{one-hot}\,(40)]$, i.e. a 229-dimensional vector (`flat_feats`).
- **Windowed** (CNN1D, CNNAttention, Transformer): a 7×43 tensor whose channels per position are $[\mathtt{avg}\,(6), \mathtt{fce}\,(21), \text{one-hot}\,(16)]$ (`windowed`).

2.3.8 Standardisation

Within each CV fold, features are z-scored with means and standard deviations computed on the training-fold rows only and applied to all rows (validation and test). For windowed input the statistics are pooled over samples and positions per channel. Because of this external scaling, the input `LayerNorm` of the PyTorch models was switched off (`normalization=False`).

2.3.9 Splits

`split()` draws 10% of all canonical groups in `4.txt` (seed 20) as the test set. This gives 1,335 test rows and 11,765 pool rows after filtering, the same set for every model and configuration. The pool is partitioned by `GroupKFold(5)` on the canonical group, so a k-mer and its reverse complement are always on the same side. For NNs, a further random 10% of each training fold (seed $100 + k$ for fold k) is the early-stopping set, and reported "train" metrics include it.

2.4 Shape feature sets benchmarked

Four feature sets were compared. Sets (a)–(c) are *shape + one-hot*: the shape windows from `data/selex` (`avg`+`fce`) concatenated with the one-hot encoding. Set (d) is *one-hot only*, with no shape features (`shape=0`). It was run with the full final protocol (output `final_onehot`), not taken from the ablation. Everything else was identical: labels from `4.txt` (the shape files have none), log affinity, count ≥10, per-strand rows, per-fold standardisation, and the same test split and grouped folds. Per window, `avg` has 6 values and `fce` 21.

Table 1: Input dimensions per feature set. Flat models: Linear, Ridge, SVR, RF, DFN. Per-position models: CNN1D, CNNAttention, Transformer. One-hot is 40 values (flat) or 16 per position. †Hexamer channels are zero at positions 0 and 6.

Feature set	Flat vector	Per-position tensor
(a) tetramer	$7 \times 27 + 40 = 229$	$7 \times (27 + 16) = 7 \times 43$
(b) hexamer	$5 \times 27 + 40 = 175$	$7 \times (27 + 16) = 7 \times 43^{\dagger}$
(c) tetramer+hexamer	$7 \times 27 + 5 \times 27 + 40 = 364$	$7 \times (27 + 27 + 16) = 7 \times 70$
(d) one-hot only	40 (10×4)	7×16 (bases of each tetramer window)

In the per-position layout, hexamer $s[j{:}j{+}6]$ ($j = 0..4$) is placed at tetramer position $j{+}1$, its central tetramer. The two end positions get zero hexamer channels, which become constant values after standardisation (`lib.windowed`). Flat models concatenate the 5 hexamer windows without padding (`lib.flat_feats`). Set (a) is the chosen preprocessing; its runs are the final runs of Section 2.8.

2.5 Timing

All timings were measured on the box: 8-core Intel Xeon CPU (1 thread per core), 15 GB RAM, no GPU, PyTorch on CPU.

- Jobs ran as 8 parallel workers. NNs, Linear, Ridge and SVR used 1 thread each; RF used 6 threads (`n_jobs`=6). Workers competed for cores, so times are indicative and comparable within this benchmark.
- *Feature time* is building the feature matrix for all 13,100 rows.
- *Training time* per run is the wall time of `fit` (including early stopping) plus one prediction pass over all rows.
- *Inference* is a separate `predict` on the 1,335 test rows, scaled to 1,000 sequences.
- The original tetramer runs did not log timing. They were re-timed by rerunning the same code with seed 0 for every fold (5 runs per model) under the same parallel setup. Their reported total is the per-run mean times the number of runs.

2.6 Validation protocol

Every k-mer is grouped with its reverse complement. A fixed random 10% of groups (seed 20, drawn over all groups in `4.txt`) forms the held-out test set. The remaining pool is split by `GroupKFold` into 5 folds. Inside each training fold, neural networks hold out a random 10% for early stopping. For every fold model we report train-fold, validation-fold and test metrics. Metrics are R^2, Pearson and Spearman correlations on log affinity, given as mean±sd over folds (and seeds), plus the train–test R^2 gap. The "averaged-runs" test R^2 is the R^2 of the mean prediction of all fold/seed models of one type.

2.7 Preprocessing ablation

Starting from a base configuration (log label, count $\geq$10, per-strand labels, tetramer avg+fce, standardisation), we changed one factor at a time and scored a random forest (300 trees, min leaf 3, max features 0.33) and a CNN1D (one seed). The factors were: label transform (raw vs log), count threshold (1, 3, 10, 30), merged vs per-strand labels, feature tables (tetramer, hexamer, both), `avg`/`fce`/both, one-hot ± shape, and standardisation. Merged labels average the measured strands per canonical k-mer and emit both orientations (reverse-complement augmentation). Per-strand labels keep each measured orientation as its own row. To make configurations comparable, every one was scored on the same rows: held-out-fold rows with a measured per-strand label and count $\geq$10. The CNN was not run at thresholds 1 and 3 because of CPU cost.

2.8 Models

The repository models are used through their `dnaffinity.models` classes. The architectures below are taken from that code (repository `PieterZanders/DNAffinity`, branch `deep-learning`). Parameter counts are for our 229-feature / 7×43 inputs.

Shared PyTorch training loop (`torch_base.TorchRegressor`).

- AdamW optimiser, MSE loss (Huber optional) and gradient-norm clipping at 5.
- ReduceLROnPlateau (factor 0.5, patience $\lfloor$patience/3$\rfloor$).
- Early stopping on the validation loss, with the best weights restored.
- Shuffled mini-batches; batches of size 1 are skipped because of BatchNorm.
- Our settings: batch size 64, at most 300 epochs, patience 25, weight decay 10^{-2}, 3 seeds (0, 1, 2) per fold.

2.8.1 Linear regression

`dnaffinity.models.Linear` wraps scikit-learn `LinearRegression` (ordinary least squares, no penalty).

- *Pros:* convex, deterministic and fast. The coefficients can be read directly.
- *Cons:* it cannot represent interactions or non-linear dependence on shape.
- *Biological relevance:* on the one-hot block it is an additive, position-independent model, equivalent to a position weight matrix (PWM) on log affinity. On the shape block it is an additive model of base-step geometry and stiffness.
- *Observed:* it underfits (train R^2 0.50, test 0.47, gap 0.03). Additivity explains about half of the variance of log affinity.

2.8.2 Ridge regression

`dnaffinity.models.Ridge` wraps scikit-learn `Ridge` (L2 penalty). The repository default is $\alpha = 1$; we used $\alpha = 10$.

- *Pros:* the penalty stabilises the strongly collinear shape features (overlapping windows, correlated parameters).
- *Cons:* same additive limit as linear regression.
- *Observed:* practically identical to linear regression (test 0.470). With 11.7k rows and 229 features the variance of OLS is already small, so the limitation is bias, not variance.

2.8.3 Support vector regression

`dnaffinity.models.SVR` wraps scikit-learn `SVR` with the repository defaults: RBF kernel, $C = 1$, $\varepsilon = 0.1$, $\gamma =$ 'scale', on standardised inputs and targets.

- *Pros:* a smooth non-linear kernel model that captures interactions implicitly and is robust thanks to the ε-insensitive loss.
- *Cons:* cost grows between quadratically and cubically with the number of samples. It is not interpretable, and the isotropic RBF kernel treats all 229 features alike.
- *Biological relevance:* similarity in kernel space means sequences with similar shape and base profiles get similar affinity, a nearest-neighbour view of the binding landscape.
- *Observed:* test 0.598, gap 0.146, better than RF.

2.8.4 Random forest

`dnaffinity.models.RandomForest` wraps scikit-learn `RandomForestRegressor` (repository defaults: 200 trees, `max_features`=1.0). We used 500 trees, `min_samples_leaf`=3 and `max_features`=0.33.

- *Pros:* non-parametric and indifferent to feature scale. It gives feature importances that can rank shape parameters per window.
- *Cons:* axis-aligned splits approximate smooth effects poorly. Deep trees memorise noisy labels.
- *Biological relevance:* threshold rules on single shape parameters (e.g. "roll at step 4 above x") are directly readable as indirect-readout hypotheses [2].
- *Observed:* largest gap (train 0.884, test 0.572). It is also the model that benefited most from shape on top of one-hot in the ablation (0.563 → 0.599), because the shape tables give trees ready-made smooth physical covariates.

2.8.5 DFN (deep feedforward network)

`dfn.DFN` builds `FeedForward`: `nn.Linear` layers with `ReLU` and `Dropout` after every hidden layer and no activation on the output. It is wrapped in `RegressionCV`, which adds an optional input `LayerNorm` (disabled here). The repository default layers are `[64,32,1]` (and `[256,128,64,32,1]` with lr 10^{-4} for SELEX in `common.py`). We used `[256,128,64,1]` (229→256→128→64→1, 100,097 parameters), dropout 0.2 and lr 3×10^{-4}.

- *Pros:* a universal approximator on the flat vector that can learn any interaction.
- *Cons:* no positional inductive bias (a motif shifted by one window is a new pattern), so it needs more data and overfits easily.
- *Observed:* test 0.644, gap 0.176, best epoch 44 of 70, the best flat model.

2.8.6 CNN1D

`cnn1d.CNN1DRegressor` builds `CNN1D` (60,481 parameters):

- optional input `LayerNorm` (off);
- stem `Conv1d`(43→64, kernel 3, padding 1) + `BatchNorm1d` + `ReLU`;
- 2 `ResidualConvBlocks`, each Conv1d(64,64,3)–BN–ReLU–Conv1d(64,64,3)–BN, a skip connection, then ReLU;
- global average pooling over the 7 positions;
- `MLPHead`: Linear(64→32)–ReLU–Dropout–Linear(32→1).

We used the repository defaults (hidden 64, 2 blocks, kernel 3, average pooling) with dropout 0.2 and lr 10^{-3}.

- *Pros:* weight sharing gives translation equivariance, so the same filter detects a feature at every position. Capacity is moderate.
- *Cons:* the receptive field is limited by depth (here the 2 blocks plus stem cover the whole 7-window input). Pooling discards absolute position.
- *Biological relevance:* first-layer filters over one-hot channels act like PWMs/motif detectors, and stacked convolutions combine adjacent sub-motifs. Pooling encodes the idea that a site can sit at any offset in the 10-mer, which matters here because the top k-mers are shifted overlaps of the same motif.
- *Observed:* best single model (test 0.736, 0.788 for averaged runs). It overfits after epoch ≈27 (final train MSE 0.068 vs. best validation 0.261), which early stopping controls. Shape channels were redundant given one-hot: 7 windows with 16 one-hot channels already determine the tetramer, so the network can learn any per-tetramer lookup itself.

2.8.7 CNNAttention

`cnn_attention.CNNAttentionRegressor` uses the same `ConvBackbone` as CNN1D, followed by `num_attn_layers`=1 `nn.MultiheadAttention` layer (embedding 64, 4 heads, dropout as configured) with a residual connection and `LayerNorm`, then global average pooling and the same `MLPHead` (77,249 parameters; repository defaults otherwise).

- *Pros:* attention can couple distant positions, beyond the convolutional receptive field, and its weights can be inspected.
- *Cons:* more parameters, and little extra range on a length-7 input.
- *Biological relevance:* pairwise attention can represent dependencies between non-adjacent positions (dinucleotide/epistatic effects, spacing of half-sites), which a PWM ignores.
- *Observed:* test 0.725, gap 0.155, slightly below CNN1D. For 10-mers the convolutions already see the whole sequence, so attention adds capacity but no new range.

2.8.8 Transformer

`transformer.TransformerRegressor` builds the following (19,073 parameters):

- optional input `LayerNorm` (off) and a `Linear` projection 43→ d_{model};
- sinusoidal positional encoding and a learnable CLS token prepended;

- `nn.TransformerEncoder` with pre-norm layers (`norm_first=True`, GELU activation);
- pooling over the CLS token or the mean of the position tokens;
- an `MLPHead` as above.

We used the `common.py` defaults for this model: $d_{\text{model}} = 32$, 2 layers, 4 heads, feed-forward 64, mean pooling, lr 3×10^{-4} with 5 warm-up epochs. The code comment notes that CLS pooling at lr 10^{-3} collapsed on SELEX. We used dropout 0.2.

- *Pros:* global, content-dependent interactions between all positions, with explicit position information.
- *Cons:* weak locality bias (no built-in notion that adjacent bases matter most). It needs much data and careful optimisation, and was the slowest to train here.
- *Biological relevance:* like attention above, it can model position-specific coupling and orientation-dependent context. Explicit positional encoding can in principle use the absolute-position and strand effects behind the label asymmetry.
- *Observed:* test 0.685, smallest gap (0.077), train MSE about equal to validation MSE (0.293 vs. 0.291), best epoch ≈112. It underfits relative to the CNNs at this size, consistent with its weaker inductive bias on about 10k training rows.

2.8.9 Averaged runs

For each model type, the "averaged-runs" prediction is the mean of the test-set predictions of all its fold×seed models (5 for non-NN models, 15 for NNs). Models of different types are never combined.

- *Pros:* averaging reduces the variance from seeds and data subsets. This is the main reason CNN1D rises from 0.736 to 0.788.
- *Cons:* the training cost is multiplied by the number of runs, and so is the inference cost when deployed.

2.8.10 Shape features and biophysics

The `avg` and `fce` channels encode indirect readout: the intrinsic geometry and deformability of each base step, which affect the energetic cost of the DNA conformation a TF binds [2]. They help models without a positional, sequence-aware inductive bias (RF: +0.036 over one-hot alone). For CNN1D they are a deterministic function of the one-hot input and gave no gain. All shape tables are reverse-complement symmetric (shift and tilt change sign), so they cannot by themselves explain the observed strand asymmetry of the labels. Only orientation-aware inputs (per-strand rows, position-specific channels) can represent it.

3 Results

3.1 Characterisation of the new shape tables

The tables are complete (all k-mers, no missing values). The `avg` files are tab-separated with a `SEQUENCE` header, and the `fce` files are space-separated. Reverse-complement symmetry holds: correlation ≥ 0.998 between a k-mer and its reverse complement, with shift and tilt sign-flipped. The new tetramer values differ from the earlier `data/input` tables. Correlations per average parameter are 0.83 (shift), 0.97, 0.96, 0.85 (tilt), 0.98 and 0.94, and 0.90–0.97 for the stiffness diagonal. Hexamer averages correlate 0.96–0.99 with those of their central tetramer, so they are

largely redundant. In `4.txt`, log affinities of the forward and reverse-complement orientations of the same 10-mer correlate only $r = 0.417$ (17,376 pairs). The correlation stays at 0.412 and 0.425 when both counts are ≥10 or ≥30, so the asymmetry is not explained by counting noise.

3.2 Preprocessing ablation

Table 2: Preprocessing ablation: 5-fold grouped CV R^2 on log affinity, evaluated on per-strand count≥10 rows (mean±sd over folds where given). Base: log, count≥10, per-strand, tetramer avg+fce, standardised.

Change vs. base	RF	CNN1D
Base	0.588±0.015	0.692±0.018
Raw label	0.579	0.691
Count ≥1 / ≥3 / ≥30	−1.449 / 0.044 / −0.227	– / – / 0.158
Merged labels + RC augmentation	0.438	0.520
Hexamer only / tetramer+hexamer	0.546 / 0.579	0.578 / 0.672
avg only / fce only	0.560 / 0.586	0.661 / 0.680
No standardisation	0.588	0.653
+ one-hot	**0.599±0.014**	0.738±0.022
One-hot only (no shape)	0.563	**0.739±0.018**
+ one-hot, tetramer+hexamer	0.589	0.721

The low and high count thresholds give strongly negative or near-zero R^2. Their Pearson correlations stay at 0.67–0.76, so the loss comes mainly from biased predicted levels: low-count labels shrink predictions, and a threshold of 30 removes low-affinity training examples. Per-strand labels beat merged labels by 0.15–0.17. The chosen configuration is log label, count ≥10, per-strand, tetramer avg+fce plus one-hot, standardised.

3.3 Model comparison per feature set

Tables 3–6 give the full metrics of every model for each of the four feature sets. Set (a), tetramer shape + one-hot, is the preprocessing chosen in the ablation.

Table 3: Feature set (a) tetramer shape + one-hot: all models, same labels, folds and held-out test set. R^2/Pearson/Spearman on log affinity, mean±sd over runs (5 folds; ×3 seeds for NNs). Gap = train − test R^2. Avg-runs = test R^2 of the averaged prediction of all runs of the model. Bold: best test R^2 in this set.

Model	Runs	Train R^2	Val R^2	Test R^2	Test Pearson	Test Spearman	Gap	Avg-runs R^2
Linear	5	0.502±0.002	0.480±0.010	0.471±0.001	0.687±0.001	0.692±0.001	0.031±0.003	0.473
Ridge	5	0.500±0.002	0.480±0.010	0.470±0.001	0.686±0.001	0.692±0.001	0.030±0.003	0.472
SVR	5	0.743±0.002	0.624±0.007	0.598±0.006	0.774±0.004	0.767±0.004	0.146±0.005	0.606
RF	5	0.884±0.001	0.599±0.015	0.572±0.004	0.762±0.003	0.744±0.003	0.311±0.003	0.581
DFN	15	0.820±0.026	0.660±0.011	0.644±0.011	0.803±0.007	0.791±0.008	0.176±0.020	0.673
CNN1D	15	0.889±0.032	0.735±0.015	**0.736±0.012**	0.860±0.006	0.844±0.007	0.153±0.022	0.788
CNNAttention	15	0.880±0.029	0.728±0.019	0.725±0.016	0.853±0.009	0.837±0.010	0.155±0.017	0.778
Transformer	15	0.761±0.018	0.705±0.015	0.685±0.009	0.828±0.006	0.813±0.006	0.077±0.010	0.709

Table 4: Feature set (b) hexamer shape + one-hot: all models, same labels, folds and held-out test set. R^2/Pearson/Spearman on log affinity, mean±sd over runs (5 folds; ×3 seeds for NNs). Gap = train − test R^2. Avg-runs = test R^2 of the averaged prediction of all runs of the model. Bold: best test R^2 in this set.

Model	Runs	Train R^2	Val R^2	Test R^2	Test Pearson	Test Spearman	Gap	Avg-runs R^2
Linear	5	0.474±0.002	0.459±0.011	0.430±0.003	0.656±0.002	0.668±0.001	0.045±0.004	0.431
Ridge	5	0.474±0.002	0.459±0.011	0.431±0.002	0.657±0.001	0.669±0.001	0.042±0.004	0.432
SVR	5	0.725±0.002	0.613±0.009	0.586±0.006	0.767±0.004	0.766±0.004	0.138±0.004	0.594
RF	5	0.881±0.002	0.576±0.017	0.549±0.006	0.745±0.004	0.726±0.004	0.332±0.005	0.558
DFN	15	0.813±0.032	0.652±0.015	0.633±0.016	0.796±0.010	0.785±0.010	0.180±0.020	0.663
CNN1D	15	0.876±0.024	0.717±0.021	**0.714±0.009**	0.847±0.005	0.830±0.006	0.162±0.020	0.765
CNNAttention	15	0.866±0.045	0.709±0.018	0.702±0.016	0.840±0.009	0.825±0.009	0.164±0.036	0.758
Transformer	15	0.763±0.013	0.700±0.012	0.673±0.012	0.821±0.007	0.807±0.008	0.089±0.012	0.699

Table 5: Feature set (c) tetramer+hexamer shape + one-hot: all models, same labels, folds and held-out test set. R^2/Pearson/Spearman on log affinity, mean±sd over runs (5 folds; ×3 seeds for NNs). Gap = train − test R^2. Avg-runs = test R^2 of the averaged prediction of all runs of the model. Bold: best test R^2 in this set.

Model	Runs	Train R^2	Val R^2	Test R^2	Test Pearson	Test Spearman	Gap	Avg-runs R^2
Linear	5	0.533±0.003	0.500±0.014	0.484±0.003	0.696±0.002	0.702±0.002	0.049±0.005	0.487
Ridge	5	0.529±0.003	0.501±0.012	0.483±0.002	0.696±0.001	0.701±0.001	0.046±0.004	0.486
SVR	5	0.730±0.002	0.616±0.007	0.588±0.006	0.768±0.004	0.764±0.004	0.142±0.005	0.596
RF	5	0.893±0.001	0.589±0.018	0.563±0.005	0.755±0.003	0.736±0.003	0.330±0.004	0.572
DFN	15	0.819±0.028	0.657±0.013	0.641±0.009	0.801±0.006	0.789±0.008	0.178±0.022	0.670
CNN1D	15	0.880±0.035	0.722±0.019	**0.723±0.013**	0.852±0.008	0.836±0.008	0.157±0.026	0.773
CNNAttention	15	0.864±0.046	0.713±0.012	0.709±0.015	0.844±0.009	0.830±0.009	0.156±0.037	0.763
Transformer	15	0.762±0.017	0.699±0.012	0.681±0.014	0.826±0.008	0.812±0.009	0.081±0.009	0.707

Table 6: Feature set (d) one-hot only (no shape): all models, same labels, folds and held-out test set. R^2/Pearson/Spearman on log affinity, mean±sd over runs (5 folds; ×3 seeds for NNs). Gap = train − test R^2. Avg-runs = test R^2 of the averaged prediction of all runs of the model. Bold: best test R^2 in this set.

Model	Runs	Train R^2	Val R^2	Test R^2	Test Pearson	Test Spearman	Gap	Avg-runs R^2
Linear	5	0.401±0.003	0.398±0.011	0.380±0.001	0.616±0.001	0.634±0.001	0.021±0.003	0.380
Ridge	5	0.401±0.003	0.398±0.011	0.380±0.001	0.616±0.000	0.634±0.001	0.021±0.003	0.380
SVR	5	0.732±0.002	0.603±0.005	0.581±0.005	0.763±0.003	0.759±0.003	0.150±0.003	0.590
RF	5	0.745±0.003	0.564±0.011	0.536±0.005	0.746±0.002	0.737±0.002	0.209±0.004	0.541
DFN	15	0.810±0.022	0.656±0.012	0.635±0.012	0.798±0.007	0.785±0.007	0.176±0.021	0.663
CNN1D	15	0.897±0.023	0.742±0.022	**0.739±0.010**	0.861±0.005	0.843±0.006	0.159±0.020	0.789
CNNAttention	15	0.909±0.025	0.733±0.021	0.729±0.009	0.856±0.005	0.838±0.005	0.180±0.023	0.785
Transformer	15	0.757±0.017	0.703±0.013	0.679±0.012	0.825±0.007	0.809±0.009	0.078±0.007	0.702

Table 7: Training dynamics of the neural networks with feature set (a) (means over runs; MSE on standardised log affinity).

Model	Best epoch	Epochs run	Final train MSE	Best val MSE
DFN	44.3	70.3	0.186	0.331
CNN1D	27.1	53.1	0.068	0.261
CNNAttention	21.6	47.6	0.055	0.266
Transformer	111.6	137.6	0.293	0.291

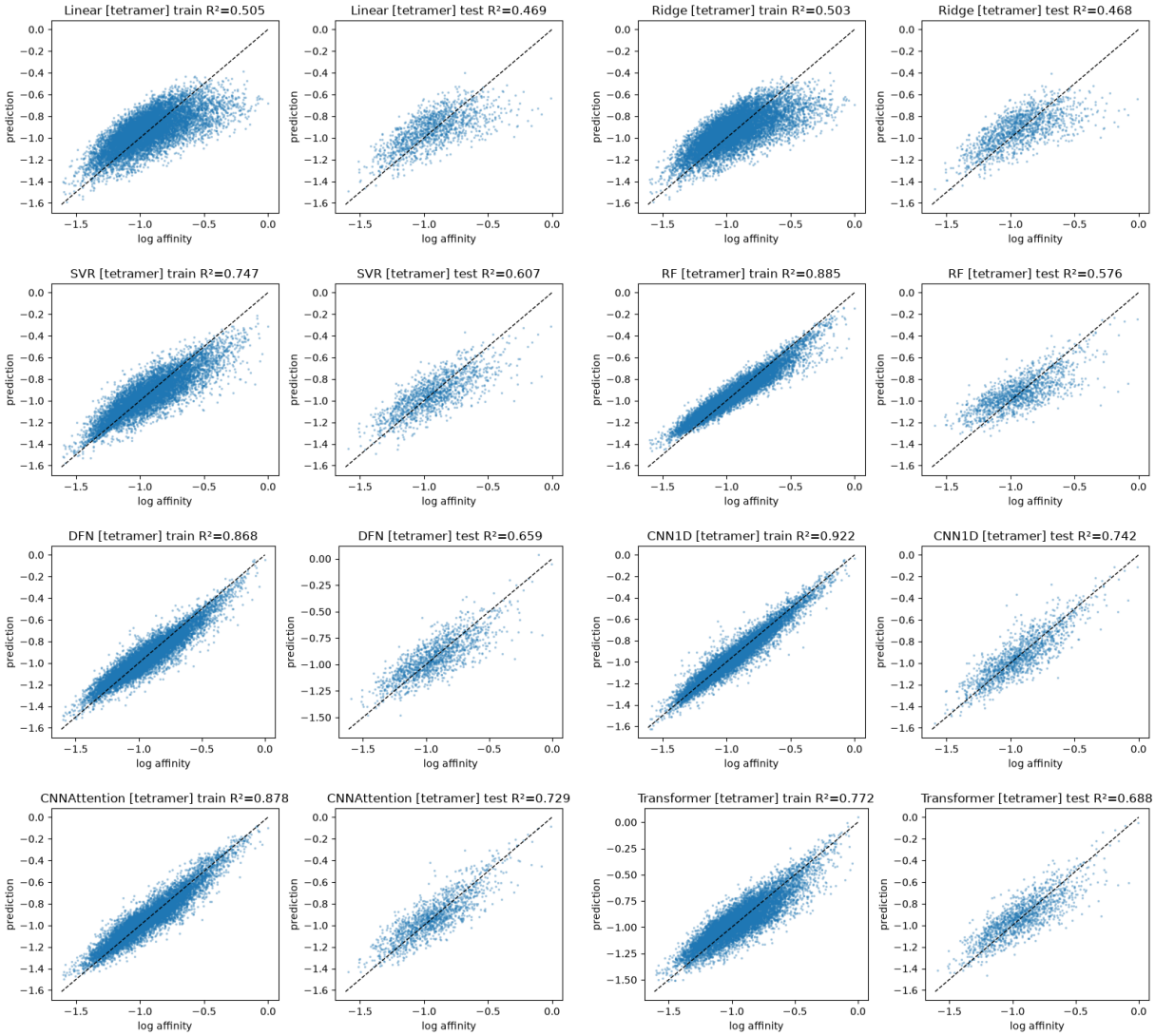


Figure 2: Ground truth vs. prediction (fold 0, seed 0) for each model with feature set (a) on its training fold (left panel) and on the held-out test set (right panel), with R^2.

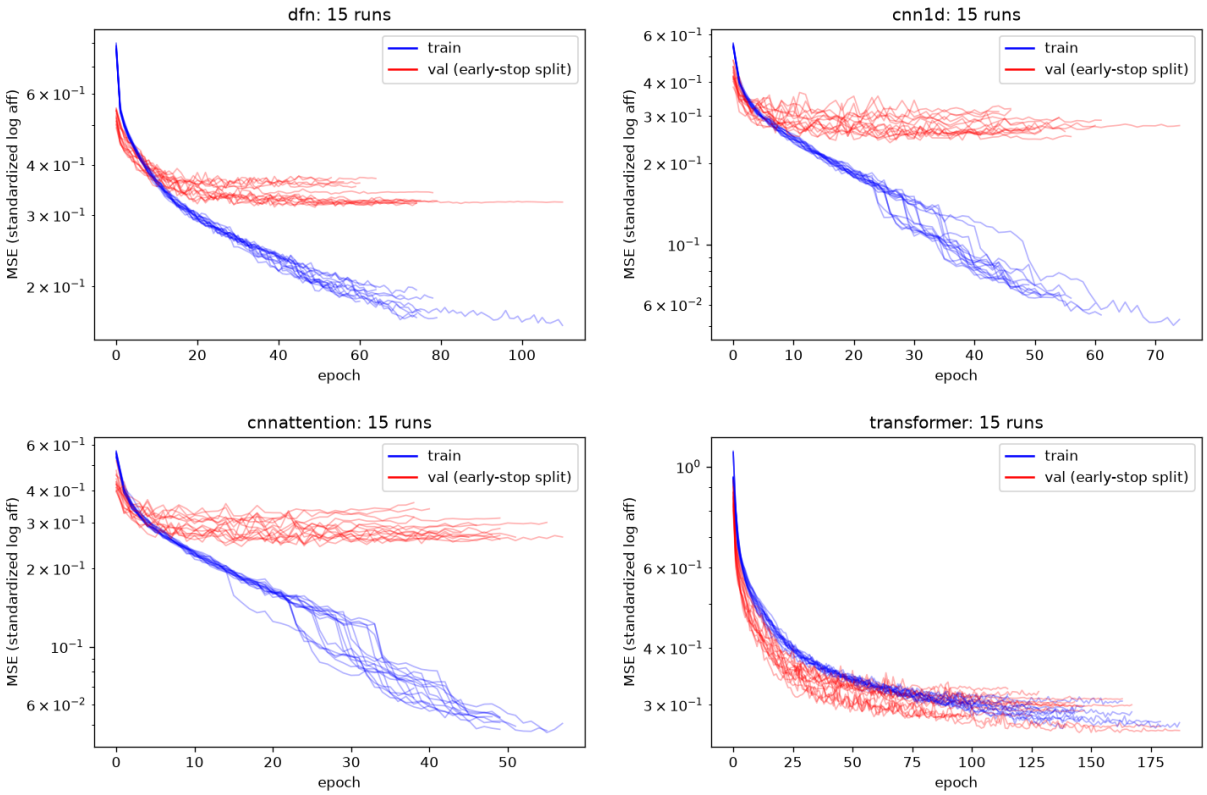


Figure 3: Learning curves of all neural-network runs with feature set (a) (blue: training loss, red: early-stopping validation loss).

3.4 Feature-set benchmark: summary

Table 8: Test R^2 summary per model and feature set (full metrics in Tables 3–6). Left block: mean over runs; right block: averaged-runs R^2. (a)–(c) are shape + one-hot; (d) is one-hot only. Bold: best feature set per model (mean test R^2).

	Test R^2 (mean over runs)				Averaged-runs test R^2			
Model	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)
Linear	0.471	0.430	**0.484**	0.380	0.473	0.431	0.487	0.380
Ridge	0.470	0.431	**0.483**	0.380	0.472	0.432	0.486	0.380
SVR	**0.598**	0.586	0.588	0.581	0.606	0.594	0.596	0.590
RF	**0.572**	0.549	0.563	0.536	0.581	0.558	0.572	0.541
DFN	**0.644**	0.633	0.641	0.635	0.673	0.663	0.670	0.663
CNN1D	0.736	0.714	0.723	**0.739**	0.788	0.765	0.773	0.789
CNNAttention	0.725	0.702	0.709	**0.729**	0.778	0.758	0.763	0.785
Transformer	**0.685**	0.673	0.681	0.679	0.709	0.699	0.707	0.702

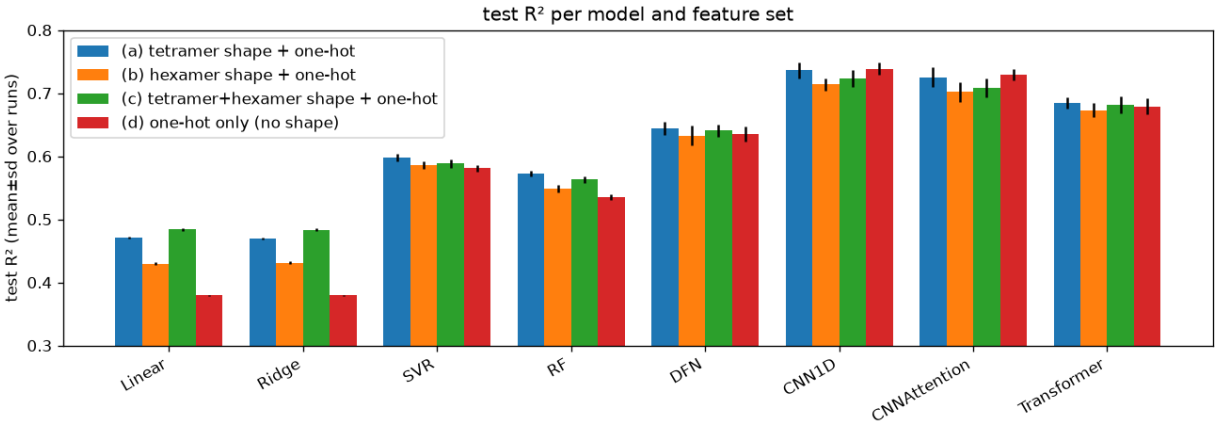


Figure 4: Test R^2 (mean±sd over runs) per model and feature set.

3.5 Processing time

Table 9: Processing time per model and feature set: feature building for all 13,100 rows [s], training time per run [s] (mean±sd), and inference [s per 1,000 sequences; 0.00 means <0.005 s]. Tetramer timings come from the re-timing runs (seed 0, 5 folds); the other sets were timed on all their runs. Hardware and threading are described in the timing subsection of the Methods.

	(a) tetramer			(b) hexamer			(c) tetra+hexa			(d) one-hot only		
Model	feat	train/run	inf/1k	feat	train/run	inf/1k	feat	train/run	inf/1k	feat	train/run	inf/1k
Linear	0.42	8.4±0.9	0.00	0.27	6.0±0.5	0.00	0.55	13.1±1.3	0.02	0.03	0.4±0.1	0.00
Ridge	0.57	0.6±0.0	0.00	0.12	0.3±0.3	0.00	0.46	0.9±0.2	0.02	0.03	0.0±0.0	0.00
SVR	0.39	58.6±7.3	2.98	0.50	62.1±10.0	3.19	0.20	84.5±19.0	4.00	0.03	10.6±0.8	0.48
RF	0.44	42.6±2.5	0.21	0.12	46.6±3.6	0.29	0.91	80.6±12.5	0.18	0.03	2.3±0.3	0.10
DFN	0.11	17.6±4.3	0.00	0.10	17.1±3.4	0.00	0.17	17.4±3.3	0.00	0.03	15.8±2.4	0.00
CNN1D	0.36	50.0±8.1	0.01	0.47	49.0±5.9	0.01	0.47	46.5±5.9	0.01	0.19	43.6±5.7	0.01
CNNAttention	0.35	55.4±8.2	0.06	0.74	76.2±16.3	0.06	0.49	55.7±10.9	0.04	0.23	58.2±6.1	0.04
Transformer	0.34	134.6±22.4	0.02	0.85	329.5±60.6	0.04	0.51	131.0±29.9	0.02	0.42	136.1±30.3	0.02

Table 10: Total training time [min] for the full protocol (5 runs for non-NN models, 15 for NNs) = mean time per run × number of runs.

Model	(a) tetramer	(b) hexamer	(c) tetra+hexa	(d) one-hot only
Linear	0.7	0.5	1.1	0.0
Ridge	0.1	0.0	0.1	0.0
SVR	4.9	5.2	7.0	0.9
RF	3.6	3.9	6.7	0.2
DFN	4.4	4.3	4.4	4.0
CNN1D	12.5	12.2	11.6	10.9
CNNAttention	13.9	19.1	13.9	14.5
Transformer	33.7	82.4	32.7	34.0

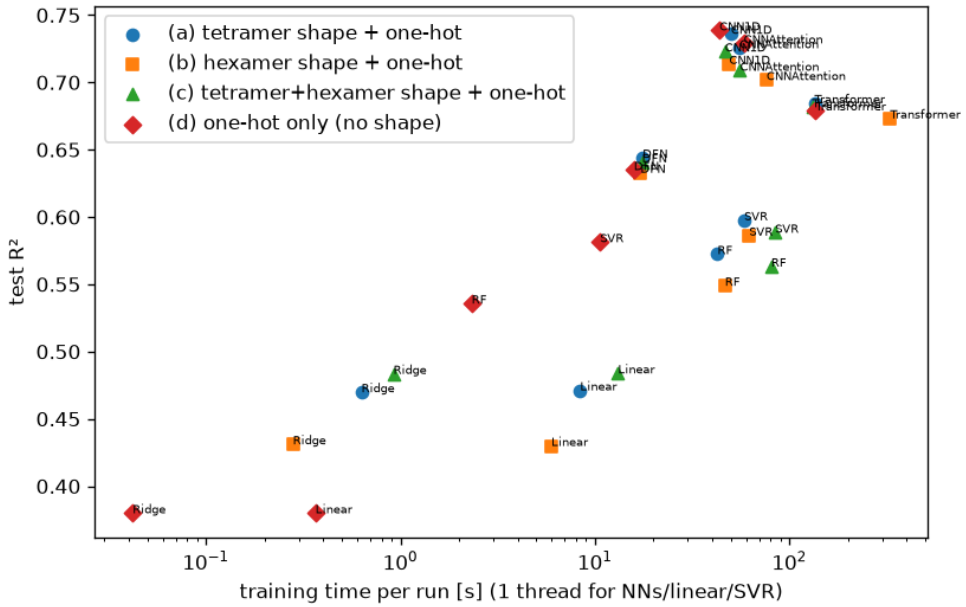


Figure 5: Training time per run vs. test R^2 for every model and feature set.

4 Discussion

Strand asymmetry. The two orientations of the same 10-mer correlate only $r \approx 0.42$ in log affinity, and this does not improve when only high-count pairs are kept (0.425 at count $\geq$30). The asymmetry is therefore systematic, not counting noise. A likely cause is the orientation-dependent expected counts of the Markov background model, possibly combined with position or flank effects in the selection library. Averaging the two strands destroys this signal. Per-strand labels with reverse-complement-grouped CV improved CV R^2 by 0.15 (RF) and 0.17 (CNN1D) over merged labels.

Count threshold. Counts below 10 add many labels whose relative SE is $\sqrt{2/n}$, i.e. $> 45\%$. A threshold of 30 removes most low-affinity k-mers and biases predictions upwards on the shared evaluation rows. A threshold of 10 was the best trade-off. The log transform was marginally better than raw affinity.

Value of the shape features. The one-hot-only set (d), run with the full final protocol, isolates the contribution of shape (Table 8). Adding tetramer shape (a) to one-hot helps exactly the models that lack a positional, sequence-aware inductive bias:

- Linear/Ridge: 0.380 $\rightarrow$ 0.471/0.470 (+0.09), the largest gain. A shape value is a fixed non-linear function of four bases, which an additive model cannot build from one-hot itself.
- RF: 0.536 $\rightarrow$ 0.572 (+0.036); its train R^2 also rises (0.745 $\rightarrow$ 0.884), as the continuous shape covariates offer many more split points.
- SVR: 0.581 $\rightarrow$ 0.598 (+0.017); DFN: 0.635 $\rightarrow$ 0.644 (+0.009); Transformer: 0.679 $\rightarrow$ 0.685 (+0.006).
- CNN1D: 0.739 (one-hot) vs. 0.736 (a); CNNAttention 0.729 vs. 0.725. The differences are within one sd over runs (0.009–0.016), so shape adds nothing for the convolutional models: with 16 one-hot channels per tetramer window they can learn any per-tetramer lookup, including the shape tables, themselves. Averaged-runs R^2 agrees (CNN1D 0.789 vs. 0.788; CNNAttention 0.785 vs. 0.778).

This confirms the ablation (CNN 0.739 vs. 0.738; RF 0.563 vs. 0.599) under the final protocol with 3 seeds. In the ablation, shape alone was informative (CNN 0.692 without one-hot) and `fce` carried more signal than `avg`.

Model comparison and overfitting.

- Linear and ridge regression underfit (train R^2 0.50, gap 0.03 with set (a); 0.40 and 0.02 with one-hot only).
- Random forest memorised the noisy labels (train 0.884, gap 0.31 with set (a)).
- SVR generalised better (gap 0.15) and was the best classical model (0.598).
- The convolutional models were best on every feature set. Their validation loss reached its minimum early (best epoch ≈22–27) while the training loss kept falling (final train MSE 0.05–0.07 vs. best validation MSE 0.26–0.27). This is overfitting, controlled by early stopping: validation and test R^2 agree closely (CNN1D 0.735 vs. 0.736), so there is no evidence of leakage.
- The Transformer had the smallest gap (0.077–0.089) and nearly equal train and validation MSE (0.293 vs. 0.291). It trained longer (best epoch ≈112) and stayed underfit relative to the CNNs.
- Averaging the runs of one model over folds and seeds helped the NNs substantially (CNN1D 0.736 → 0.788, CNNAttention 0.725 → 0.778, DFN 0.644 → 0.673), and the non-NN models only slightly (+0.002–0.009).

Which shape feature set? Among the shape sets, tetramer (a) gave the highest test R^2 for every non-linear model (Table 8):

- CNN1D: 0.736 vs. 0.714 (hexamer) and 0.723 (both);
- CNNAttention: 0.725 / 0.702 / 0.709;
- Transformer: 0.685 / 0.673 / 0.681;
- DFN: 0.644 / 0.633 / 0.641;
- SVR: 0.598 / 0.586 / 0.588;
- RF: 0.572 / 0.549 / 0.563.

Only the additive models preferred the combined set (Linear 0.484 and Ridge 0.483 vs. 0.471/0.470 with tetramers). Hexamers were the worst shape set for every model; for the CNNs, DFN and Transformer they were even below one-hot only. There are three reasons:

1. Hexamer averages are 96–99% correlated with their central tetramer, so they add little new information.
2. A 10-mer has only 5 hexamer windows against 7 tetramer windows, so the outermost base steps are not described (for windowed models the end positions are zero-padded).
3. The combined set doubles the shape channels (364-dimensional flat input, 70 channels per position) without adding signal, which slightly increases variance for the flexible models.

The additive models gain about 0.013 from the combination because a hexamer value acts as a fixed non-linear (context-dependent) feature of 6 bases. The differences between feature sets (0.01–0.02 for non-linear models) are small compared with the differences between models, but they are consistent across models and comparable to or larger than the sd over runs.

Computational cost. Ridge trains in under 1 s per run. SVR, RF, DFN and the CNNs take about 0.3–1.5 min per run under the parallel load, and the Transformer about 2–5.5 min. Feature building takes under 1 s in all cases. Inference is $\leq$0.3 s per 1,000 sequences for all models except SVR ($\approx$3–4 s with shape, 0.5 s with one-hot only), whose cost scales with the number of support vectors and the input dimension. Dropping shape makes the classical models much cheaper (SVR 58.6 $\rightarrow$ 10.6 s, RF 42.6 $\rightarrow$ 2.3 s per run), whereas NN training time barely changes (CNN1D 50.0 vs. 43.6 s). CNN1D on one-hot only therefore gives the best accuracy at moderate cost (Figure 5). The Transformer hexamer runs (mean 330 s) overlapped with the heaviest parallel load, so part of that difference reflects contention.

SELEX_test.txt mismatch. Only 8.4% of the 5,511 10-mers in `SELEX_test.txt` appear in `4.txt`. On the 462 shared k-mers its values are uncorrelated with the `4.txt` affinities (Pearson of logs -0.03, Spearman -0.06). Its scale is different (max 1.0, weights 1 or 10). It probably comes from another experiment or processing, so it was not used as a test set.

Limitations. The ablation used a single seed and one-factor-at-a-time changes. We did not test de-duplication of shifted motif overlaps, or similarity-based grouping beyond reverse complements, so overlapping k-mers sharing a core motif can still occur on both sides of a split. Hyperparameters were not tuned per model. The only test data come from the same SELEX experiment.

5 Conclusion

The new MD shape tables are complete, internally consistent and differ measurably from the earlier tetramer tables. For Gata4 HT-SELEX the optimal preprocessing is log affinity, count $\geq$10, per-strand labels, tetramer avg+fce plus one-hot, and standardisation, evaluated with reverse-complement-grouped CV. With this setup CNN1D reaches a held-out test R^2 of 0.736 (0.788 when its runs are averaged) and is the best of the eight models, ahead of CNNAttention (0.725), the Transformer (0.685), DFN (0.644), SVR (0.598), RF (0.572) and Linear/Ridge (0.47). The four-way feature-set benchmark shows:

- Shape features are redundant for the convolutional networks: one-hot only gives the same accuracy (CNN1D 0.739, CNNAttention 0.729) at slightly lower cost.
- Shape features clearly help models without a positional sequence bias: Linear/Ridge +0.09, RF +0.036, SVR +0.017, DFN +0.009 over one-hot only.
- Tetramer tables are the best shape set for every non-linear model; hexamers alone are consistently worst, and combining them with tetramers helps only the linear models (+0.013), consistent with their near-redundancy with the central tetramer.

CNN1D is the best accuracy/cost trade-off (about 45–50 s per run on one CPU thread). The main open issues are the strong strand asymmetry of the SELEX labels and the lack of an independent test set consistent with `4.txt`.

A Per-feature-set diagnostic plots

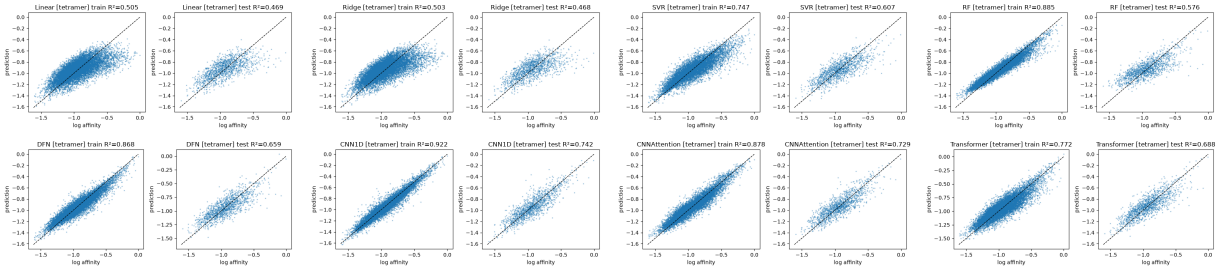


Figure 6: Ground truth vs. prediction (fold 0, seed 0; left half of each panel: train, right half: test) for the tetramer shape + one-hot feature set.

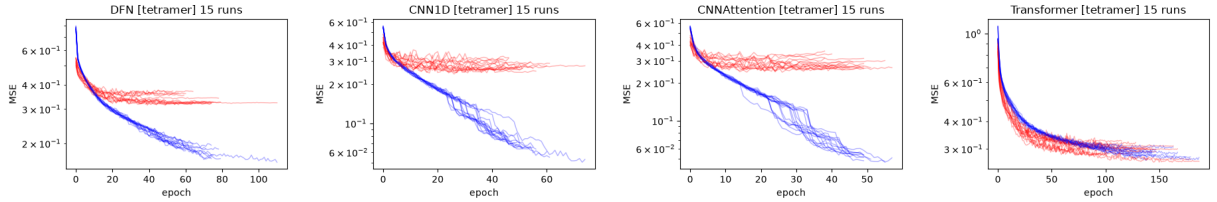


Figure 7: Learning curves (blue: train, red: early-stopping validation; 15 runs each) for the tetramer shape + one-hot feature set.

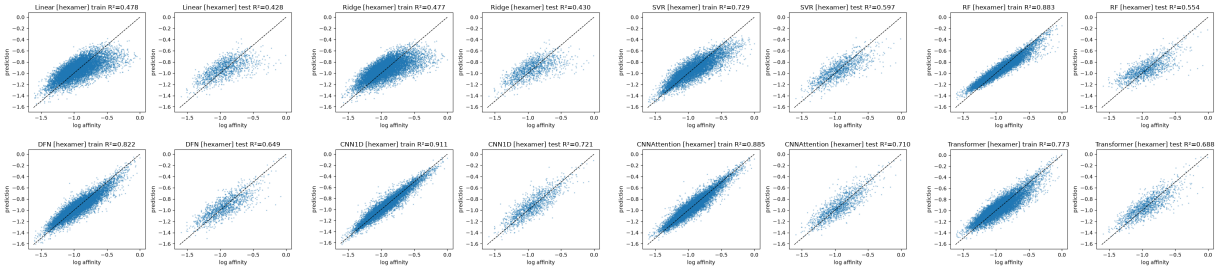


Figure 8: Ground truth vs. prediction (fold 0, seed 0; left half of each panel: train, right half: test) for the hexamer shape + one-hot feature set.

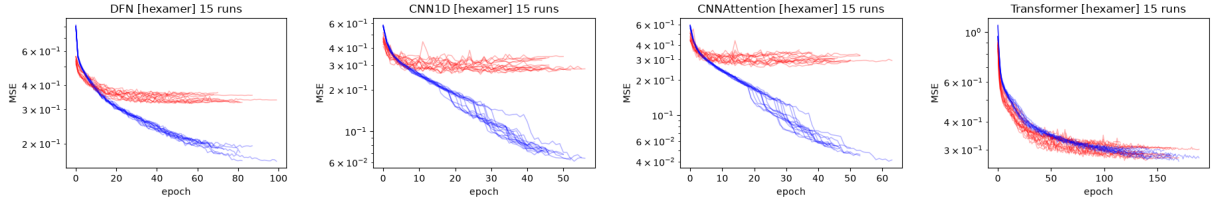


Figure 9: Learning curves (blue: train, red: early-stopping validation; 15 runs each) for the hexamer shape + one-hot feature set.

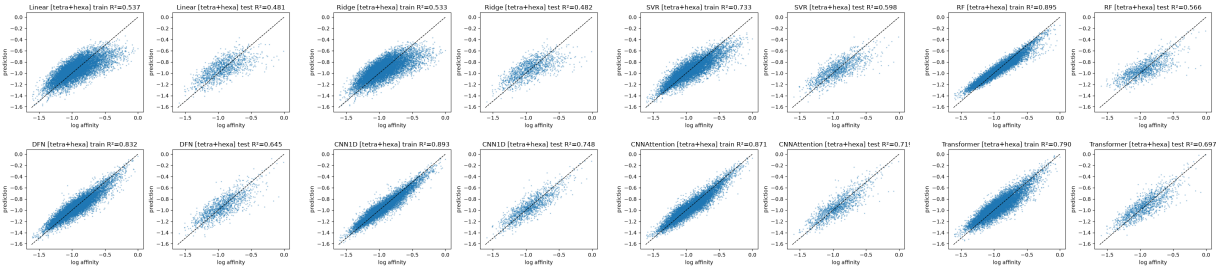


Figure 10: Ground truth vs. prediction (fold 0, seed 0; left half of each panel: train, right half: test) for the tetramer+hexamer shape + one-hot feature set.

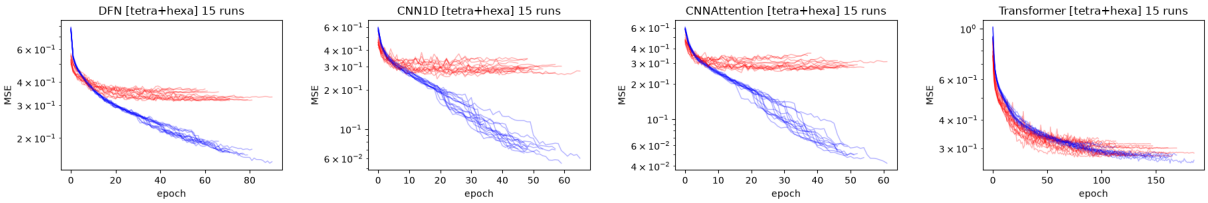


Figure 11: Learning curves (blue: train, red: early-stopping validation; 15 runs each) for the tetramer+hexamer shape + one-hot feature set.

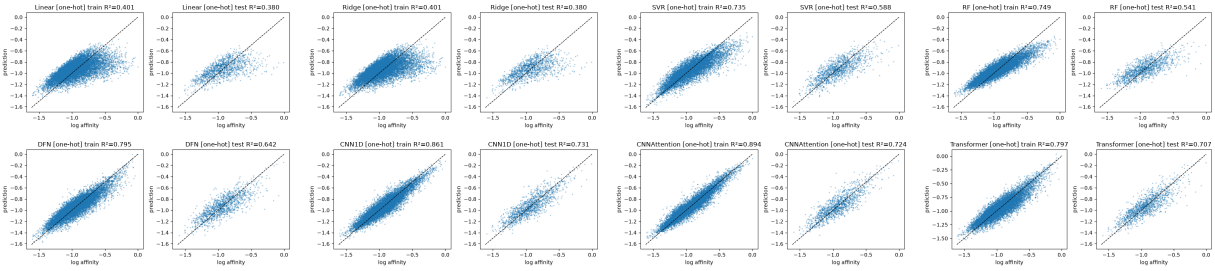


Figure 12: Ground truth vs. prediction (fold 0, seed 0; left half of each panel: train, right half: test) for the one-hot only feature set.

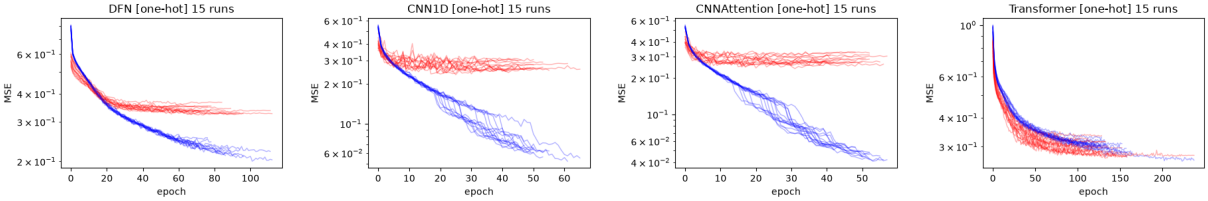


Figure 13: Learning curves (blue: train, red: early-stopping validation; 15 runs each) for the one-hot only feature set.

References

[1] A. Jolma *et al.*, DNA-binding specificities of human transcription factors, *Cell* 152, 327–339 (2013).

[2] R. Rohs *et al.*, The role of DNA shape in protein–DNA recognition, *Nature* 461, 1248–1253 (2009).